

FIT5222 Pacman Challenge

Submission Instruction

Hi everyone, we are glad to announce that the contest server for the FIT5222 Pacman Challenge is online at <https://fit5222a2.contest.pathfinding.ai/>! Now, you can submit your implementation and battle with your peers. Read the submission instructions before submission!

Evaluation

Beating the Baseline Implementation

The baseline implementation (staff implementation) is `staffTeam.py`.

To be eligible for placement on the leaderboard (and to compete against your peers), your implementation must convincingly beat the baseline (win 28/49 games).

Note that the `staffTeam` on your local machine is the exact same as that on the leaderboard—this means you are able to work on the assignment locally. Remember to test your performance across multiple games and maps!

You are expected to clearly win against the staff implementation. Failing to beat the baseline means that your grade for Criteria 1 of the marking rubric will be a **Fail**.

If your submission wins convincingly, it will be eligible for head-to-head matches and participation in tournaments.

Head-to-Head Matches

Each head-to-head is a single 'Match', meaning that you play 49 games against another competitor. The result of each match against another team is qualitatively classified as:

- A convincing win (win at least 28 out of 49 games).
- A tie (neither team scores a convincing win).
- A loss (lose 28 games or more).

Due to the computational burden for all-pairs evaluation, timely evaluation may not be possible if we pit your implementation against all other competitors. Instead, we'll use a

series of head-to-head matches to determine an ELO rating (EP) to give you a strong idea where you stand on the leaderboard.

Upon submission (and beating the baseline), your entry immediately enters head-to-head evaluations. Your team will be matchmade across a minimum of 22 matches against other submissions of similar rating in order to seed your position on the leaderboard. Win matches to gain EP; scaling with how convincing the win is, and the relative strength of the opponent. This rating may later fluctuate as you either beat, tie, or lose to submissions from other students.

This should provide you with a strong idea of your submission's performance until we compute your true score in **tournaments**.

Tournaments

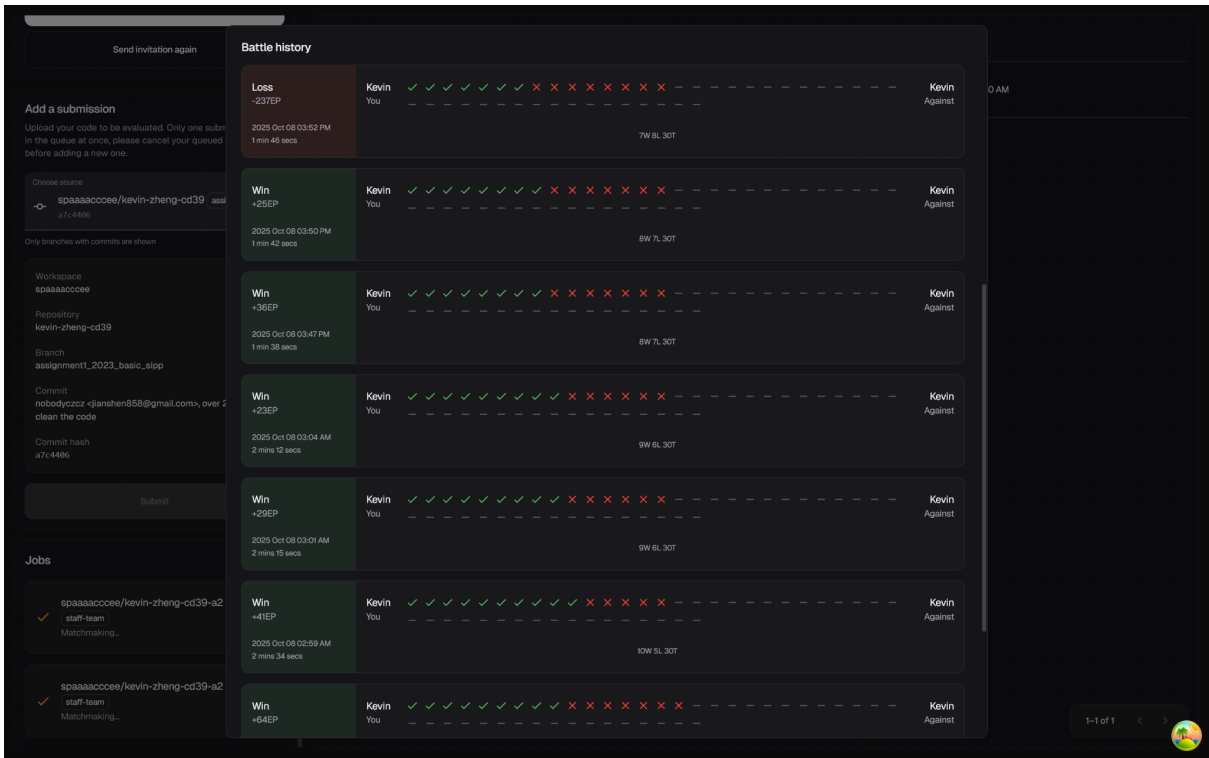
Your true score will be determined by your victory points (VP) in tournaments. At set times, the server will pause regular submissions in order to compute a full all-pair evaluation of the leaderboard (meaning your team will be fully evaluated against all other players). Afterwards, you will have an exact leaderboard ranking that is a true representation of your score (and subsequent marks). The times for these checkpoint computations will be announced via Ed. A final computation will be made at the assignment deadline. Each match against another team is determined as follows:

- In the event of a convincing win (win at least 28 out of 49 games), you gain 3 VP.
- In the event of a tie (neither team scores a convincing win), you gain 0.5 VP.
- In the event of a loss (lose 28 games or more), you gain 0 VP.

Scoring

Official leaderboard ranking is determined by your victory points (Wins/Ties) against all other competitors in **tournaments**. This will be determined at set times, and at the competition deadline (including any submission made before this cutoff).

Details



You can find each submission’s details, such as matches played, won, lost, and the number of match wins/ties/victory points in Jobs.

Deadline

Submissions made *before* the Due time of Assignment 2 are considered on-time (even if evaluation has not yet completed). Submissions made after the deadline are considered late and will apply late penalties per day after the deadline.

Restrictions

Submission count

You may only have **up to 2** ‘live’ submissions at once. These are submissions that are either pending evaluation or ready to battle against other players. This agent will play in head-to-heads and tournaments on your behalf. If you’ve used up both slots, you must remove one of your existing submissions before adding another. You’ll see a warning if you attempt to remove your best performing agent.

RAM

The limitation on RAM is 2GB for each run. Normally, you won't consume that much memory.

Time

Action Time

Each timestep, an agent has a maximum of 5 seconds to return action. Use “-c” argument when running Pacman on your machine to ensure you never exceed this time limit.

Total Time Limit

Each game has an **overall time limit of the 2400s**. We expect a game to finish much earlier than this time limit (2~5 minutes). You need to ensure that your algorithm runs fast enough to avoid being killed by the system. Decisions need to be reached efficiently in a dynamic and fast paced, online environment.

Submission

The submission process is exactly the same as assignment 1. The contest website is available at fit5222a2.contest.pathfinding.ai.

Log in

Open the contest website. Click “Sign in with Bitbucket”.

If you're not automatically logged in, choose “Continue with Google”. Then, follow its prompts to log in with your Monash University credentials.

Create your repository

Click “Create repository” on the top left.

We'll create a repository for you and invite you to it. Look for an email in your inbox from Bitbucket.

After you accept the invitation, you should have access to a personal repository.

Push your code to the repository

It takes a little setup before Bitbucket lets you push your local code to it, because unlike other Git providers, Bitbucket doesn't let you use your account password to authenticate yourself. You'll need to create an API token instead, which you could use when prompted for your password.

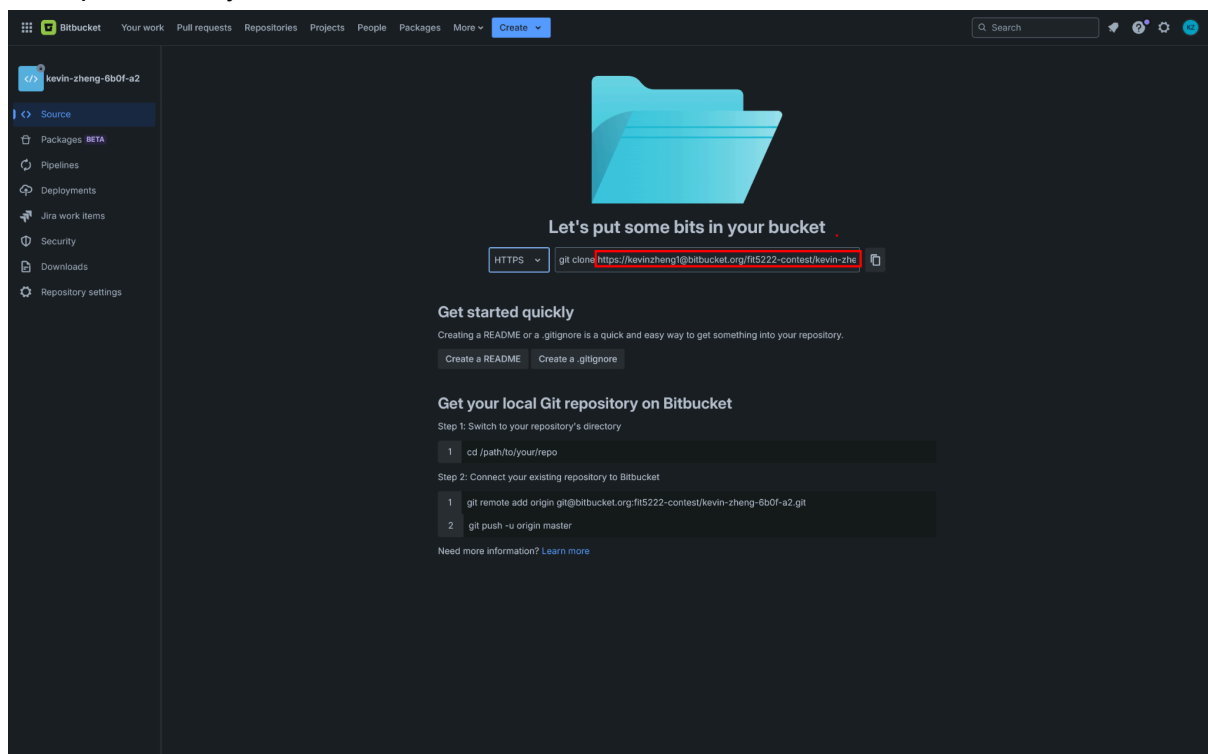
Creating an API Token

In Bitbucket, on the top right corner, choose "Atlassian account settings". Then in the Security tab, choose "Create and manage API tokens". Choose "Create a scoped API token" and follow its instructions to receive an API token. You can now use this in place of your password.

Submit your code

Adding the remote

If you haven't already, you should add your personal repository's URL to your local Git workspace. Find your URL here:



Next, in your local repository, in a terminal window, type:
`git remote add personal <your-url-here>`

Commit and push your changes

In your local repository, in a terminal window, type the following to commit your changes:

```
git add *  
git commit -m "Changed xyz"
```

Remember to write meaningful commit messages to track updates as you work (and be able to efficiently roll-back to previous strategies as desired).

Now, after you commit changes, you'll be able to push to your remote repository via:

```
git push personal <branch-name-here>
```

Submit your code to the server

When you have your code ready in a Bitbucket repository, you'll see it in the drop down here. You'll only see repositories with at least one commit. You must also have write access to those repositories. Select your desired branch and verify that the commit info is correct. Click "Submit" to add it to the submission queue.

The screenshot shows a web interface for submitting code. It has a dark theme. The 'Add a submission' section is highlighted with a red box. It contains a dropdown menu for 'Choose source' with the selected option 'spaaaacccc/kevin-zheng-cd39' and a commit hash '246f435'. Below the dropdown, it says 'Only branches with commits are shown'. Underneath, there is a table-like view showing the workspace 'spaaaacccc', repository 'kevin-zheng-cd39', branch 'testing', commit message 'Kevin <kzhe0012@student.monash.edu>, 10 days ago, Update gitignore', and commit hash '246f435'. A 'Submit' button is at the bottom of this section. The 'Jobs' section at the bottom shows 'No jobs found'.

Inspect jobs

In the "Jobs" panel, you'll see your job getting queued, eventually picked up and evaluated. You'll also see if it's currently in a battle, as well as your battle history. If your

submission errors, you can click on the job to see error messages. You can cancel queued jobs by clicking the trash icon.

Set a nickname for yourself (optional)

You can set a nickname that'll appear publicly on the leaderboard. You'll appear as an anonymous challenger by default. **NOTE: your username *must be appropriate*** in a classroom setting. Inappropriate nicknames may be penalised. If you are in doubt, choose a different nickname. We think that nicknames are a fun option of personalisation, but **this is a *privilege* that we may have to *revoke* if a select few abuse this.** **Nicknames must NOT include the word "Staff".**

Dependencies

- If you require any non-standard python package, comment on the ED discussion. We will install it for you and reply to you after it's installed.

Path

- If your `myTeam.py` need to import from other python files that are not included in standard Pacman codes, add this on the top of your codes:

```
sys.path.append(os.path.dirname(os.path.abspath(__file__)))
```
- The above will add the folder your **myTeam.py** is in to the python path, so you can easily import anything inside that folder.
- If you need to read a file, always use a relative path and relative to the **BASE_FOLDER**. For example, if you have `pacman.pddl` together with your **myTeam.py**, use

```
file_path = BASE_FOLDER+"/pacman.pddl"
```

as the path to `pacman.pddl`.

A better solution would be to always use the **os.path library** to manipulate paths.

With `os.path` library, the above path would be:

```
file_path = os.path.join(BASE_FOLDER,"pacman.pddl")
```